

Network design practice 5-2

김경민

May 28th, 2026

1 Assignment 1

```
IPv4AddressHelper ipv4;
ipv4.SetBase("10.1.1.0", "255.255.255.0");
IPv4InterfaceContainer i = ipv4.Assign(devices);

TypeId tid = TypeId::LookupByName("ns3::UdpSocketFactory");
Ptr<Socket> recvSink = Socket::CreateSocket(c.Get(0), tid); /* Assignment1 1000: add sta node container*/
InetSocketAddress local = InetSocketAddress(IPv4Address::GetAny(), 80);
recvSink->Bind(local);
recvSink->SetRecvCallback(MakeCallback(&ReceivePacket));

Ptr<Socket> source = Socket::CreateSocket(c.Get(1), tid); /* Assignment1 1000: add ap node container*/
InetSocketAddress remote = InetSocketAddress(IPv4Address("255.255.255.0"), 80);
source->SetAllowBroadcast(true);
source->Connect(remote);

// Tracing
WifiPhy.EnablePcap("wifi-security-example", devices);

// Output what we are doing
std::cout << "Security Mode: " << secMode << std::endl;
std::cout << "Testing " << numPackets << " packets sent with receiver rss " << rss << std::endl;

}

if (secMode == "WPA2-PSK")
{
    // Compute crypto keys from OpenSSL
    std::cout << "\n[WPA2-PSK] Running OpenSSL crypto..." << std::endl;
    CryptoResult cr = wpa2_derive_keys(passphrase, ssid,
                                     anonc, snonce,
                                     ap_mac, sta_mac); /* Assignment1 1000: add passphrase and ssid */

    std::cout << " PMK (PBKDF2-SHA1, 4096 iter): "
               << bytes_to_hex(cr.pmk, 16) << " "
               << " (" << std::fixed << std::setprecision(3)
               << cr.pmk_ms << " ms)" << std::endl;
    socket, pktsize, numPackets, interval);
}
else if (secMode == "WPA3-SAE")
{
    std::cout << "\n[WPA3-SAE] Running OpenSSL crypto..." << std::endl;
    CryptoResult cr = wpa3_derive_keys(passphrase, ssid,
                                     anonc, snonce,
                                     ap_mac, sta_mac); /* Assignment1 1000: add passphrase and ssid */

    std::cout << " PMK (SAE/ECC P-256 ECDH): "
               << bytes_to_hex(cr.pmk, 16) << " "
               << " (" << std::fixed << std::setprecision(3)
```

Figure 1: Code

Filled in passphrase and ssid variable. Created sockets on Node 0 and 1, so that the UDP server application can receive data. Filled in the passphrase, the length of the passphrase, ssid, and the length of the ssid.

```

received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$ ./ns3 run "comnet
-5-2-example --secMode=None --numPackets=4 --packetSize=100 "
Security Mode: None
Testing 4 packets sent with receiver rss -80
Received one packet!
Received one packet!
Received one packet!
Received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$

```

Figure 2: secMode=None, original settings

Run the code with secMode set to None. Observed successful build and all four packets received.

2 Assignment 2

```

Received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$ ./ns3 run "comnet
-5-2-example --secMode=WPA2-PSK --numPackets=4 --packetSize=100 "
Security Mode: WPA2-PSK
Testing 4 packets sent with receiver rss -80

[WPA2-PSK] Running OpenSSL crypto...
PMK (PBKDF2-SHA1, 4096 iter): 2891a4556474d686c599018ffe1caff1.
.. (2.850 ms)
PTK (PRF-512/HMAC-SHA1): a133b8b51e87dc33a158502f886e5fe3.
.. (0.006 ms)
AES-CCMP per packet: 10.06 us
Crypto delay: 2.86 ms → handshake starts at t=2ms

--- WPA2 4-Way Handshake Phase ---
[0.002s] 4-Way HS Msg1: AP -> STA (ANonce)
[0.012s] 4-Way HS Msg2: STA -> AP (SNonce + MIC)
[0.022s] 4-Way HS Msg3: AP -> STA (GTK + MIC)
[0.032s] 4-Way HS Msg4: STA -> AP (ACK) -- Handshake Complete
Received one packet!
Received one packet!
Received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$

```

Figure 3: secMode=WPA2-PSK

In assignment 1, secMode is set to None. The Access point and station establish a connection instantly with zero cryptographic overhead. The four packets are received immediately. When secMode is set to WPA2-PSK, it uses pairwise master key for the pairwise transient key. 4-way handshake can be observed. Data packets are sent only after this handshake concludes. It takes 0.032ms to complete the task.

3 Assignment 3

```
Received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$ ./ns3 run "connet
-5-2-example --secMode=WPA3-SAE --numPackets=4 --packetSize=100 "
Security Mode: WPA3-SAE
Testing 4 packets sent with receiver rss -80

[WPA3-SAE] Running OpenSSL crypto...
PMK (SAE/ECC P-256 ECDH): ccaa91ae51a9b3ae6ccc0f1a35eeb5a2..
. (0.340 ms)
PTK (HMAC-SHA256 KDF): 526fe5b6ba5687de19ddf930b9bcb356.
.. (0.003 ms)
AES-CCMP per packet: 2.73 us
Crypto delay: 0.34 ms → handshake starts at t=0ms

--- WPA3 SAE + 4-Way Handshake Phase ---
[0.000s] SAE Commit: STA -> AP
[0.010s] SAE Commit: AP -> STA
[0.020s] SAE Confirm: STA -> AP
[0.030s] SAE Confirm: AP -> STA -- PMK Established
[0.040s] 4-Way HS Msg1: AP -> STA (ANonce)
[0.050s] 4-Way HS Msg2: STA -> AP (SNonce + MIC)
[0.060s] 4-Way HS Msg3: AP -> STA (GTK + MIC)
[0.070s] 4-Way HS Msg4: STA -> AP (ACK) -- Handshake Complete
Received one packet!
Received one packet!
Received one packet!
ns3@ubuntu:~/ns-0-RAN-flexric/mmwave-LENA-oran$
```

Figure 4: secMode=WPA3-SAE

Now, the secMode is set to WPA3-SAE. It takes extra step for complex authentication phase before the 4-way handshake. Elliptic curve cryptography is used for the PMK and PTK. It achieves stronger security by initializing SAE commit and confirm phase. The total connection time takes 0.07ms, which is slower compared to 0.032ms whose of WPA2.

WPA2 uses the Wi-Fi password and SSID to derive a master key. They exchange Nonces, combine these with the PMK to derive PTK, and then exchange MIC to verify that the both sides computed the same key.

WPA3 complements the vulnerability of WPA2 by introducing a SAE phase before a 4-way handshake. Both devices commit to a secret value derived from the password using elliptic curve cryptography, and confirms each other's proofs. This extra step makes WPA3 immune to offline dictionary attacks, preventing potential decryption risks.